Architecture:

```
User types question
      ↓
Streamlit UI
      ↓
Django API (POST /ask/)
      ↓
LangGraph Graph
↓                  ↓
Planner  →       Researcher
Agent            Agent
          ↓
 MCP Tool (mock search)
          ↓
       Final Answer
          ↓
  Saved to PostgreSQL
          ↓
  Shown in Streamlit
```

Multi Agent workflow

User → Planner Agent → Research Agent → Tool → Answer

Planner Agent

- Thinks
- Decides what to do
- Does Not fetch data
- Structured Output

Research Agent

- Executes Plan
- Calls tools
- Returns data
- Deterministic execution

MCP - model context protocol provides structured way for AI agents to use tools.

Mental Model - Research Agent → HTTP call → MCP Tool → Returns JSON

Phase 1 - MCP TOOL

Create mcp/server.py

CODE:

```
from fastapi import FastAPI                    // imports FastAPI framework

app = FastAPI()                                // Creates an API app instance

@app.get("/search")                            // {GET /search} request
def search_web(query: str):                    // The endpoint expects {/search?query=something}
    mock_data = {
        "capital of France": "Paris is the capital of France.",
        "capital of Germany": "Berlin is the capital of Germany.",
        "capital of India": "Delhi is the capital of India."
    }

    return {
        "result": mock_data.get(query, "No data found.")        // Returns JSON
    }
```

Phase 2 - Planner Agent

The planner converts the user query into structured machine instruction, this is important because the Research agent will parse it using string matching.

Create agents/prompts.py

CODE: (Testing phase)

```
PLANNER_PROMPT = """
You are a planning assistant.

Your job is to decide what information is needed
to answer the user's question.

If research is needed, respond EXACTLY in this format:
SEARCH: <query>

If you already know the answer, respond EXACTLY in this format:
ANSWER: <final answer>
```

```
Do not add extra text.                    //This reduces hallucination.
"""
```

Create agents/planner_agent.py

CODE: (Testing phase)

```
from langchain_ollama import ChatOllama
from .prompts import PLANNER_PROMPT

llm = ChatOllama(
    model="tinyllama",
    temperature=0
)

def planner_node(state):
    question = state["question"]

    response = llm.invoke(
        PLANNER_PROMPT + "\nUser Question: " + question
    )

    return {
        "plan": response.content
    }
```

Download ollama and pull tinyllama
pip install langchain-ollama

Create agents/test_planner.py

CODE: (Testing phase)

```
from .planner_agent import planner_node

if __name__ == "__main__":
    state = {"question": "What is the capital of France?"}
    result = planner_node(state)
    print(result)
```

Now run the test_planner by writing python -m agents.test_planner

The output - {'plan': 'Your job is to decide what information is needed to answer the user\'s question, and to provide it in a clear and concise manner. If research is needed, respond

EXACTLY in this format: SEARCH: "What is the capital of France?"\n\nIf you already know the answer, respond EXACTLY in this format: ANSWER: Paris, France'}

As We can see the model is failing to follow instructions properly i.e instead of outputting :
SEARCH: capital of France

It outputs: the instructions again and an example answer

Thus, we need to replace tinyllama with Llama3 as tinyllama is struggling with strict formatting and instruction following (prompting).

Now, in the powershell write ollama pull llama3
In planner_agent.py replace the model from tinyllama to llama3

CODE :

```
llm = ChatOllama(
    model="llama3",
    temperature=0
)
```

Now we need to hardcode the planner prompt inside the planner_agent.py instead of a different file prompt.py (i deleted the file)

```
PLANNER_PROMPT = """
You are a planning assistant.

If the question requires information lookup:
Respond EXACTLY:

SEARCH: <search query>

Do not answer directly.
Return only one line.
```

So now the updated planner_agent.py look like:

CODE :

```
from langchain_ollama import ChatOllama
from langchain_core.messages import SystemMessage, HumanMessage

llm = ChatOllama(
    model="llama3",
```

```
    temperature=0
)

PLANNER_PROMPT = """
You are a planning assistant.

If the question requires information lookup:
Respond EXACTLY:

SEARCH: <search query>

Do not answer directly.
Return only one line.
“”” 

def planner_node(state):
    question = state["question"]

    messages = [
        SystemMessage(content=PLANNER_PROMPT),
        HumanMessage(content=question)
    ]

    response = llm.invoke(messages)

    output = response.content.strip()

    # Guardrail enforcement
    if not output.startswith("SEARCH:"):
        output = f"SEARCH: {question}"

    return {
        "plan": output
    }
```

Now run the test_planner by writing python -m agents.test_planner

OUTPUT : {'plan': 'SEARCH: Paris capital'} which is better than previous test cases

Now, before implementing the research agent, i added the calculator tool exposed via MCP

Planner → "CALCULATE: 23 * 7"
Research → calls MCP calculator tool
MCP → executes calculation

Create a mcp/calculator_tool.py

CODE:

```
import ast
import operator

# Supported operators
OPERATORS = {
    ast.Add: operator.add,
    ast.Sub: operator.sub,
    ast.Mult: operator.mul,
    ast.Div: operator.truediv,
    ast.Mod: operator.mod,
    ast.Pow: operator.pow,
}

def safe_eval(node):
    if isinstance(node, ast.Constant):  # numbers
        return node.value

    elif isinstance(node, ast.BinOp):
        left = safe_eval(node.left)
        right = safe_eval(node.right)
        op_type = type(node.op)

        if op_type in OPERATORS:
            return OPERATORS[op_type](left, right)
        else:
            raise ValueError("Unsupported operator")

    else:
        raise ValueError("Invalid expression")


def calculate(expression: str):
    try:
        tree = ast.parse(expression, mode='eval')
        result = safe_eval(tree.body)
        return {
            "status": "success",
            "result": result
        }
```

```
    except Exception as e:
        return {
            "status": "error",
            "message": str(e)
        }
```

Update the server.py (MCP Server)

CODE:

```
from fastapi import FastAPI
from calculator_tool import calculate

app = FastAPI()

@app.get("/search")
def search_web(query: str):
    mock_data = {
        "capital of France": "Paris is the capital of France.",
        "capital of Germany": "Berlin is the capital of Germany.",
        "capital of India": "Delhi is the capital of India."
    }

    return {
        "result": mock_data.get(query, "No data found.")
    }

@app.post("/calculate")
def calculator_endpoint(data: dict):
    expression = data.get("expression")
    return calculate(expression)
```

Now open a terminal inside the MCP directory and run this command
- uvicorn server:app --reload --port 8001

If an error comes : 'uvicorn' is not recognized as an internal or external command, operable program or batch file.

Run this command: pip install uvicorn fastapi
And then run the previous command again
It should show this in the terminal - Uvicorn running on http://127.0.0.1:8001

Use http://127.0.0.1:8001/docs#/ (FastAPI gives you Swagger UI)

In the POST /calculate (Calculator Endpoint), Click “Try it out” and in the JSON box replace everything in the box with

```
{
  "expression": "23 * 7"
}
```

Click Execute (Note - inside the JSON box it might show something as “additonalProp1”: {} because this endpoint currently accepts a generic dict. FastAPI doesn’t know your expected schema yet.

Now that the calculator endpoint is working we need to update the planner_agent.py

CODE:

```
import re
from langchain_ollama import ChatOllama
from langchain_core.messages import SystemMessage, HumanMessage

llm = ChatOllama(
    model="llama3",
    temperature=0
)

PLANNER_PROMPT = """
You are a planning assistant.

If the question requires information lookup:
Respond EXACTLY:

SEARCH: <search query>

Do not answer directly.
Return only one line.
"""

def is_math_question(question: str):
    # Detect simple arithmetic patterns
    math_pattern = r'[\d\.\s\+\-\*\/\%]+'
    return bool(re.search(r'\d+\s*[\+\-\*\/]\s*\d+', question))


def planner_node(state):
    question = state["question"]
```

```
    # 🔹 Step 1: Rule-based math detection
    if is_math_question(question):
        # Extract expression (simple version)
        expression = re.findall(r'\d+\s*[\+\-\*\/]\s*\d+', question)
        if expression:
            return {"plan": f"CALCULATE: {expression[0]}"}

    # 🔹 Step 2: Otherwise use LLM for search planning
    messages = [
        SystemMessage(content=PLANNER_PROMPT),
        HumanMessage(content=question)
    ]

    response = llm.invoke(messages)
    output = response.content.strip()

    if not output.startswith("SEARCH:"):
        output = f"SEARCH: {question}"

    return {"plan": output}
```

And also, we need to update the test_planner.py

CODE:

```
from .planner_agent import planner_node

if __name__ == "__main__":
    test_questions = [
        "What is the capital of France?",
        "What is 23 * 7?",
        "Calculate 45 + 12"
    ]

    for question in test_questions:
        state = {"question": question}
        result = planner_node(state)

        print("Question:", question)
        print("Planner Output:", result)
        print("-" * 40)
```

Now run the test_planner by writing python -m agents.test_planner

OUTPUT -

Question: What is the capital of France?
Planner Output: {'plan': 'SEARCH: Capital of France'}

Question: What is 23 * 7?
Planner Output: {'plan': 'CALCULATE: 23 * 7'}

Question: Calculate 45 + 12
Planner Output: {'plan': 'CALCULATE: 45 + 12'}

As we can see the planner is achieving what we were looking for
Capital question → SEARCH ✅
Math question → CALCULATE ✅
Explicit calculate → CALCULATE ✅

Now we move to phase 3

Phase 3 - Research Agent

Create research_agent.py inside agents/

CODE:

```
import requests

MCP_CALCULATOR_URL = "http://127.0.0.1:8001/calculate"

def research_node(state):
    plan = state["plan"]

    if plan.startswith("CALCULATE:"):
        expression = plan.replace("CALCULATE:", "").strip()

        try:
            response = requests.post(
                MCP_CALCULATOR_URL,
                json={"expression": expression}
            )

            result = response.json()
```

```
            if result["status"] == "success":
                return {"answer": str(result["result"])}
            else:
                return {"answer": "Calculation error."}

        except Exception as e:
            return {"answer": f"MCP call failed: {str(e)}"}

    elif plan.startswith("SEARCH:"):
        query = plan.replace("SEARCH:", "").strip()
        return {"answer": f"Search tool not implemented yet: {query}"}

    else:
        return {"answer": "Invalid plan format."}
```

Update the test_planner.py to test the flow

CODE -

```
from .planner_agent import planner_node
from .research_agent import research_node

if __name__ == "__main__":
    questions = [
        "What is 23 * 7?",
        "What is the capital of France?"
    ]

    for question in questions:
        state = {"question": question}

        state.update(planner_node(state))
        state.update(research_node(state))

        print("Final State:", state)
        print("-" * 50)
```

Now before running python -m agents.test_planner we start a terminal in the MCP folder and write this command to run the MCP “uvicorn server:app --reload --port 8001”

OUTPUT -

```
(venv) C:\Users\wridi\OneDrive\Desktop\Assignment_1>python -m agents.test_planner
Final State: {'question': 'What is 23 * 7?', 'plan': 'CALCULATE: 23 * 7', 'answer': '161'}
```

Final State: {'question': 'What is the capital of France?', 'plan': 'SEARCH: Capital of France', 'answer': 'Search tool not implemented yet: Capital of France'}

Right now our system works like this -

Question → planner_node() → research_node() → Final State //Manual control of execution order.

Now, we need to refactor this manual chaining of agents to LangGraph StateGraph workflow Instead of we are manually setting the executions such as - First we run the planner agent & then we run the research agent, we define Planner → Research

Question //GOAL of this phase
↓
LangGraph
↓
Planner Node
↓
Research Node
↓
Final State

First we install LangGraph inside our virtual environment by typing this command
pip install langgraph

LangGraph requires a defined state structure, so that it can pass data between nodes safely.
Create - agents/state.py

CODE - This is our shared memory.

```
from typing import TypedDict, Optional

class AgentState(TypedDict):
    question: str                    //"question": "What is 23 * 7?",
    plan: Optional[str]              //"plan": None,
    answer: Optional[str]            //"answer": None
```

Planner updates "plan" and Research updates "answer" {LangGraph merges updates automatically}.

Now, we will create the Graph [Tutorial]
Create - agents/graph.py

CODE -

```
from langgraph.graph import StateGraph
from agents.state import AgentState
from agents.planner import planner_node
from agents.research_agent import research_node


def build_graph():

    # Create workflow with defined state
    workflow = StateGraph(AgentState)

    #  Register nodes
    workflow.add_node("planner", planner_node)
    workflow.add_node("research", research_node)

    #  Define execution order
    workflow.set_entry_point("planner")
    workflow.add_edge("planner", "research")

    #  Define where it ends
    workflow.set_finish_point("research")

    #  Compile workflow
    app = workflow.compile()

    return app
```

// StateGraph(AgentState) → we are telling LangGraph: “This workflow will pass around a dictionary shaped like AgentState”
// add_node(“planner”, planner_node) → “When we say planner, run planner_node.”
// set_entry_point(“planner”) → “Execution always starts with planner.”
// add_edge(“planner”, “research”) → “After planner finishes → go to research.”
// set_finish_point(“research”) → After research finishes → stop.
// compile() → Builds executable workflow

Now we need to test the Graph
Create - agents/test_graph.py

CODE -

```
from agents.graph import build_graph
```

```
if __name__ == "__main__":

    app = build_graph()

    result = app.invoke({
        "question": "What is 23 * 7?",
        "plan": None,
        "answer": None
    })

    print("Final State:", result)
```

Now let's run the test by typing this in console python -m agents.test_graph

OUTPUT - Final State: {'question': 'What is 23 * 7?', 'plan': 'CALCULATE: 23 * 7', 'answer': '161'}

So, we can see when we ran result = app.invoke({...})

- LangGraph created an initial state → {"question": "What is 23 * 7?", "plan": None, "answer": None}
- It executed the entry node → "planner" → planner returned : {"plan": "CALCULATE: 23 * 7"} → LangGraph merged it automatically
- It followed the edge → "research" → research returned : {"answer": "161"} → Merged again
- Reached finish point → stopped → Return final state

Now we need to implement the /search endpoint in the agents/research_agent.py

At the top, we add : MCP_SEARCH_URL = "http://127.0.0.1:8001/search"

Now replace your SEARCH block with this :

CODE -

```
elif plan.startswith("SEARCH:"):
    query = plan.replace("SEARCH:", "").strip()

    try:
        response = requests.get(
            MCP_SEARCH_URL,
            params={"query": query}
        )

        result = response.json()
```

```
        return {"answer": result["result"]}

    except Exception as e:
        return {"answer": f"Search failed: {str(e)}"}
```

What it does -

- Now the planner outputs → SEARCH: Capital of France
- Research Extracts → Capital of France
- Then sends HTTP request → GET /search?query=Capital of France
- The MCP Server receives
  ```
  @app.get("/search")
  def search_web(query: str):
  ```

Returns → {"result": "Paris is the capital of France."}
Research agent then returns → {"answer": "Paris is the capital of France."}
LangGraph merges it.

Before restarting the MCP server, we need to update the agents/test_graph.py

CODE -

```
result = app.invoke({
    "question": "What is the capital of France?",
    "plan": None,
    "answer": None
})
```

OUTPUT -

Final State: {'question': 'What is the capital of France?', 'plan': 'SEARCH: Capital of France', 'answer': 'Paris is the capital of France.'}

Now we have architecturally achieved → User Question → LangGraph → Planner Node → Research Node → MCP Server [/calculate, /search] → Final State Returned

Now, we have to build the Django backend + /ask/ endpoint.
Inside the backend/ folder write this command to create the project → `django-admin startproject core .`
And to create the API App → `python manage.py startapp api`
Open backend/core/settings.py , find INSTALLED_APPS = [..... → add 'rest_framework', 'api',
Run to test the Django Server by running this command → `python manage.py runserver`

Django is a backend web framework which helps us to receive HTTP requests from the browser/frontend, processes the logic & return responses (usually JSON for APIs).

User → Django API → LangGraph → MCP → Django → User // It acts as the middle layer that exposes our AI system to the outside world.

We want this :

```
POST /api/ask/
Body:
{
  "question": "What is 23 * 7?"
}
```

Django should receive the question → call our LangGraph workflow → return :

```
{
  "question": "...",
  "plan": "...",
  "answer": "..."
}
```

manage.py	Command center for Django
settings.py	Project configuration
core/urls.py	Main routing file
api/views.py	Where your API logic lives
api/urls.py	Routes specific to API

Now, we create the API Endpoint, backend/api/views.py

CODE -

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from rest_framework import status


@api_view(['POST'])                                  //This function only accepts POST requests
def ask_question(request):
    question = request.data.get("question")
```

```
    if not question:
        return Response(
            {"error": "Question is required."},
            status=status.HTTP_400_BAD_REQUEST
        )

    return Response({
        "message": "Received successfully",
        "question": question
    })
```

Now we have to connect the URL to This View i.e we must let django know that if someone calls /api/ask/, run ask_question.
Create backend/api/urls.py

CODE -

```
from django.urls import path
from .views import ask_question

urlpatterns = [
    path('ask/', ask_question),
]
```

Now we connect it to the main project by opening backend/core/urls.py

CODE -

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('api.urls')),
]
```

Run the command to test - python manage.py runserver

We can use swagger UI, but for this we will simply use Django Browsable API
Go to - http://127.0.0.1:8000/api/ask/

In the form/JSON editor we test by writing the query and then click on post -

```
{
  "question": "Hello"
}
```

Now we replace the dummy response and connect Django → LangGraph

First we need to import the LangGraph App, so we open backend/api/views.py
At the top of the codebase, add →

```
import sys
import os

# Make root directory visible
sys.path.append(os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
)

from agents.graph import build_graph

app = build_graph()
```

Replace the Dummy Response →

```
@api_view(['POST'])
def ask_question(request):
    question = request.data.get("question")

    if not question:
        return Response(
            {"error": "Question is required."},
            status=status.HTTP_400_BAD_REQUEST
        )

    try:
        # Call LangGraph
        result = app.invoke({
            "question": question,
            "plan": None,
            "answer": None
        })

        return Response({
            "question": result.get("question"),
            "plan": result.get("plan"),
            "answer": result.get("answer"),
        })

    except Exception as e:
        return Response(
```

```
        {"error": str(e)},
        status=status.HTTP_500_INTERNAL_SERVER_ERROR
    )
```

Restart the server by writing this command → python manage.py runserver

Test in Browser using the Django Browsable API → http://127.0.0.1:8000/api/ask/

Enter -

```
{
  "question": "What is the capital of France?"
}
```

And click POST.

OUTPUT →

```
{
  "question": "What is the capital of France?",
  "plan": "SEARCH: Capital of France",
  "answer": "Paris is the capital of France."
}
```

Now, we will use PostgreSQL to save every question, plan and answer in the database
First, we install PostgreSQL for windows, open → pgAdmin → Create DB - agentic_ai_db_assignment1

Next, we configure Django to use the PostgreSQL
Open → backend/core/settings.py

Find →

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

Replace →

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'agentic_ai_db_assignment1',
```

```
        'USER': 'postgres',
        'PASSWORD': 'your_password_here',
        'HOST': 'localhost',
        'PORT': '5432',
    }
}
```

Next, we create a model to store the conversations. Open → backend/api/models.py
Replace →

```
from django.db import models

class Conversation(models.Model):
    question = models.TextField()
    plan = models.TextField()
    answer = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.question
```

The definition of the table looks like this →

id	auto
question	text
plan	text
answer	text
created_at	timestamp

Inside the backend/ folder open a powershell and run this command to migrate the table from Django to PostgreSQL automatically.

→

```
python manage.py makemigrations
python manage.py migrate
```

Now, we save data after LangGraph Runs
Open backend/api/views.py
Import the model → `from .models import Conversation`

Now inside the backend/api/view.py , we need to modify the function by replacing the return section with →

```
result = app.invoke({
    "question": question,
    "plan": None,
    "answer": None
})

# Save to database                    //We need to add this code-snippet
Conversation.objects.create(
    question=question,
    plan=result.get("plan"),
    answer=result.get("answer"),
)

return Response({
    "question": result.get("question"),
    "plan": result.get("plan"),
    "answer": result.get("answer"),
})
```

Now, we restart the Django server by writing the command in the backend/ folder
→ python manage.py runserver //The MCP server should also be running

Test in Browser using the Django Browsable API → http://127.0.0.1:8000/api/ask/

Ask →

```
{
  "question": "What is 23 * 7?"
}
```

Now, we can check the api_conversation table inside pgAdmin

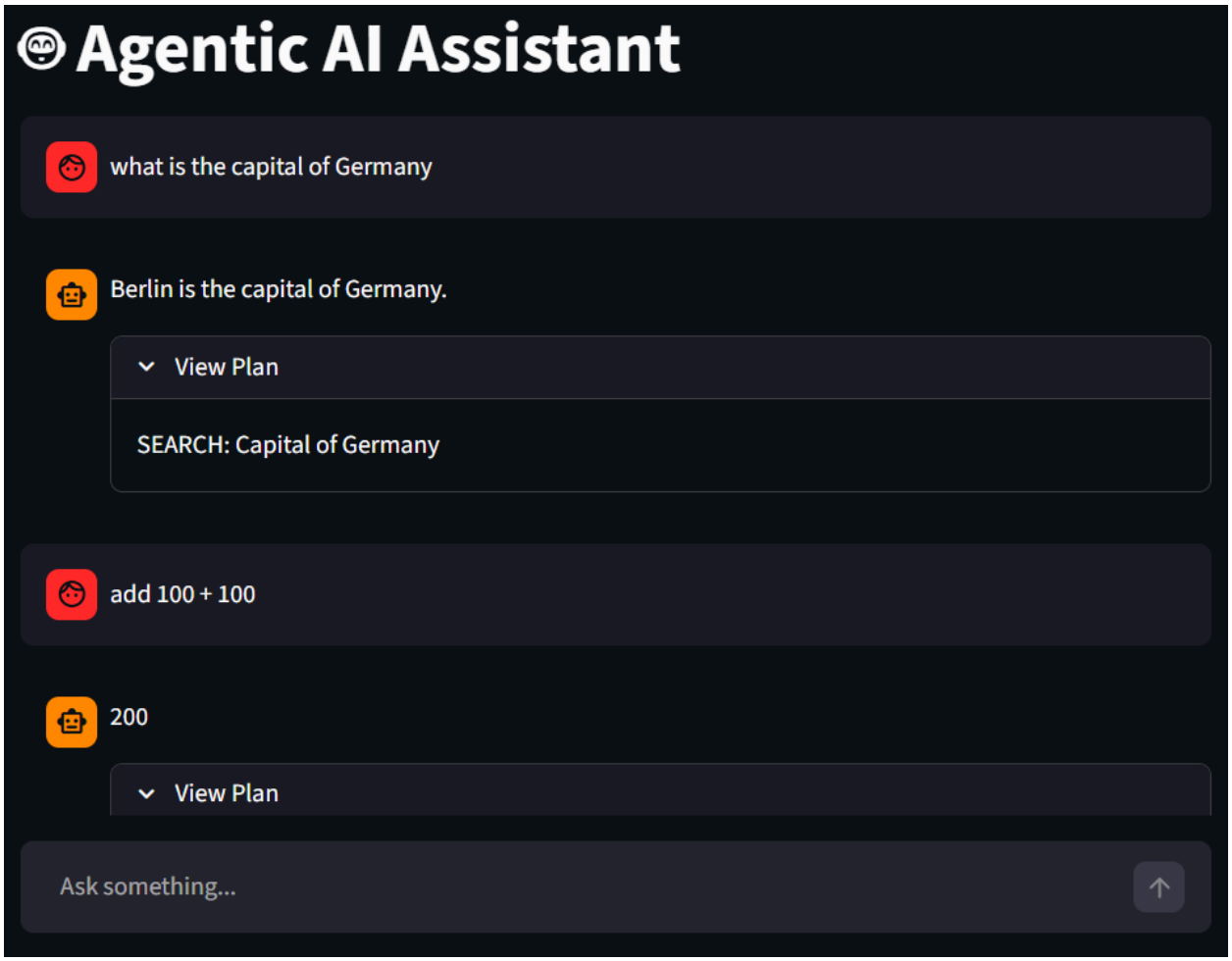
	id [PK] bigint	question text	plan text	answer text	created_at timestamp with time zone
1	1	What is 23 * 7?	CALCULATE: 23 * 7	161	2026-02-24 14:41:23.634177+05:...
2	2	What is capital of France?	SEARCH: Capital of France	Paris is the capital of France.	2026-02-24 14:59:03.789916+05:...
3	3	What is the capital of France?	SEARCH: Capital of France	Paris is the capital of France.	2026-02-24 16:01:32.792047+05:...
4	4	what is 24 * 7?	CALCULATE: 24 * 7	168	2026-02-24 16:02:44.29067+05:30
5	5	what is the capital of Germa...	SEARCH: Capital of Germa...	Berlin is the capital of Germa...	2026-02-24 16:41:45.785401+05:...
6	6	add 100 + 100	CALCULATE: 100 + 100	200	2026-02-24 16:42:38.715188+05:...

Now, we need to build the Streamlit UI (frontend)
Inside the ui/ directory, we need to create app.py

CODE -

```
import streamlit as st
import requests

API_URL = "http://127.0.0.1:8000/api/ask/"

st.set_page_config(page_title="Agentic AI System", layout="wide")

st.title("🤖 Agentic AI Assistant")

if "history" not in st.session_state:
    st.session_state.history = []

question = st.chat_input("Ask something...")

if question:
    with st.spinner("Thinking..."):
        response = requests.post(API_URL, json={"question": question})

    if response.status_code == 200:
        data = response.json()

        st.session_state.history.append({
            "question": data["question"],
            "answer": data["answer"],
            "plan": data["plan"]
        })
    else:
        st.error(response.json())

# Display chat history
for item in st.session_state.history:
    with st.chat_message("user"):
        st.write(item["question"])

    with st.chat_message("assistant"):
        st.write(item["answer"])
        with st.expander("View Plan"):
            st.write(item["plan"])
```

Now open a new terminal, activate the VENV Environment → cd ui → write this command -
streamlit run app.py


Agentic AI Assistant
what is the capital of Germany
Berlin is the capital of Germany.
View Plan
SEARCH: Capital of Germany
add 100 + 100
200
View Plan
Ask something...